EE5203 L01 Lab Guide

Build, flash, debug, and modify Blinky - Basri Erdogan

Mission: Create a Nucleo-F401RE project, make LD2 blink, prove that the program executes in the debugger, and independently change its timing.

Prepare before class

- ☐ Complete the **pre-lab quiz** in CATS (**Tests & Quizzes**): *L01 Pre-Lab Quiz (Lab 1)*, **due Tuesday 22 September at 13:00**. Two attempts; the higher score counts. Allow about 15 minutes. Part A comes from this guide and the Reference page; Parts B and D check the C background the course builds on. **Part C is binary and hexadecimal, which we have not covered or reviewed yet** - look it up and use your second attempt to learn it. Answers are released on Wednesday 23 September at 13:00, after the last lab session.
- ☐ Write out the **entry ticket** and bring it on paper: what observable change you expect after modifying the delay, what evidence shows the new firmware actually reached the board, and what you would check first if VS Code cannot detect ST-LINK. You will be asked about these at checkoff.
- ☐ Lab PCs have everything installed: log in with your own student account. On your own laptop, install the tools named in the Reference page first.
- ☐ Bring a USB **data** cable if using your own equipment.

Learning objectives

By checkoff time, you can:

1. build, flash, run, and debug a Nucleo-F401RE project;
2. distinguish build evidence from hardware evidence;
3. set a breakpoint and inspect a variable;
4. make and explain one controlled code modification.

Hardware and files

Item	Required value
Board	Nucleo-F401RE
Programmer/debugger	On-board ST-LINK USB connection
Output	LD2, connected to PA5
Tools	STM32CubeMX (configure/generate) + VS Code (edit/build/debug)
Project name	`EE5203_L01_<studentnumber>`

Session plan

Time	What happens
0:00 – 0:15	**Opening demonstration** — one live pass through the complete workflow.
0:15 →	**Checkpoints A–D** below at your own pace; the oral check happens when you reach D.

The demonstration follows this path once:

```
create -> configure -> generate -> edit -> build -> flash -> debug -> verify
```

Observe which tool owns each stage: the `.ioc` configuration lives in STM32CubeMX; `main.c`, the build terminal, debug controls, and the variable panels live in VS Code. Record questions; do not copy menu clicks without understanding the stage they belong to.

Checkpoint A - Orient: board and tools

1. Connect the board through the ST-LINK USB connector.
2. In **STM32CubeMX**: create a project for **NUCLEO-F401RE** (Board Selector) and accept the board's default peripheral initialization.
3. Confirm that PA5/LD2 is configured as a GPIO output.
4. In Project Manager set the name (no spaces), the location `C:\Work\<studentnumber>`, and **Toolchain/IDE = CMake**; generate code.
5. Copy the folder `C:\Users\Public\EE5203_template\.vscode` into the generated project folder, next to the `.ioc` file (Reference page → Workflow, *Template* row). Without it VS Code cannot build or debug.
6. Open the generated folder in **VS Code** (File → Open Folder → the folder that contains the `.ioc`; choose preset **Debug** if asked) and locate the protected `USER CODE` regions in `main.c`.
7. Build the unchanged project with **Ctrl+Shift+B**: zero errors, and a `Linking C executable` line at the end.

Evidence for Checkpoint A: Show the selected board, PA5 configuration, and a successful build message.

Checkpoint B - Prove: build and flash

Inside the **existing generated** `while (1)`, between `/* USER CODE BEGIN 3 */` and `/* USER CODE END 3 */`, add these statements. Do not create a second infinite loop.

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(500);
```

Build with **Ctrl+Shift+B**, flash with **F5** (Debug (ST-LINK)), press Continue, and verify a stable blink. Explain why the physical observation is stronger evidence than the build message alone.

Checkpoint C - Observe: breakpoint and variable

Add before the loop, between `/* USER CODE BEGIN 2 */` and `/* USER CODE END 2 */`:

```
int toggle_count = 0;
```

Increment it after every toggle:

```
toggle_count = toggle_count + 1;
```

1. Place a breakpoint on or immediately after the increment — and **write down which**: a breakpoint *on* the increment pauses before it runs, so the first value you see is `0`; on the line *after* it, the first value is `1`. Predict for the line you chose.
2. Start a debug session.
3. Predict the value before pressing **Resume**.
4. Resume twice and record the observed values.

Observation	Predicted	Observed
First pause		
Second pause		
Third pause		

Evidence for Checkpoint C: Show the breakpoint, execution marker, and changing `toggle_count` value.

Checkpoint D - Own: modify and justify

Each student must make one assigned change and demonstrate it individually.

Variant	Requirement
A	Change the delay to 100 ms
B	Change the delay to 250 ms
C	Change the delay to 750 ms
D (stretch)	Blink three times quickly, then pause for 1 s — write the three toggle pairs out; the loop that shortens this arrives in L02

Before flashing, state the predicted behavior. Change only what is necessary, then collect physical or debugger evidence.

Acceptance checklist

- ☐ Correct Nucleo-F401RE project selected.
- ☐ Project builds without errors.
- ☐ LD2 exhibits the required core behavior.
- ☐ Breakpoint is reached and `toggle_count` changes as predicted.
- ☐ Individual timing variant is demonstrated.
- ☐ Student explains build versus flash versus run evidence.
- ☐ Code remains inside protected user-code regions.

Marks (10): pre-lab quiz + entry ticket 2 · Blinky 3 · debugger evidence 1 · modification 2 · oral check 2.

Individual oral check

Be prepared for one question and one live change:

- What does a successful build prove?
- What does ST-LINK do?
- Why does the breakpoint pause before or after a particular statement?
- Where should application code be placed, and why?
- What evidence proves your modified firmware is running?

Submit

Submit the requested source/project, one debugger screenshot, the completed prediction table, and a two-sentence debugging report if a failure occurred.

If you are stuck

Report expected and observed behavior; read the **first relevant** error; confirm the project, board, ST-LINK, build, and flash; test whether a breakpoint reaches the loop; then show the TA your evidence and smallest next test.